

LaTCoder: Converting Webpage Design to Code with Layout-as-Thought

Yi Gui*
Huazhong University of Science and Technology
Wuhan, China

Zhen Li*†
Huazhong University of Science and Technology
Wuhan, China

Zhongyi Zhang†
Huazhong University of Science and Technology
Wuhan, China

Guohao Wang†
Huazhong University of Science and Technology
Wuhan, China

Tianpeng Lv
Huazhong University of Science and Technology
Wuhan, China

Gaoyang Jiang
Huazhong University of Science and Technology
Wuhan, China

Yi Liu
Huazhong University of Science and Technology
Wuhan, China

Dongping Chen
Huazhong University of Science and Technology
Wuhan, China

Yao Wan*‡
Huazhong University of Science and Technology
Wuhan, China

Hongyu Zhang
Chongqing University
Chongqing, China

Wenbin Jiang*
Huazhong University of Science and Technology
Wuhan, China

Xuanhua Shi*
Huazhong University of Science and Technology
Wuhan, China

Hai Jin*
Huazhong University of Science and Technology
Wuhan, China

Abstract

Converting webpage designs into code (design-to-code) plays a vital role in *User Interface* (UI) development for front-end developers, bridging the gap between visual design and functional implementation. While recent *Multimodal Large Language Models* (MLLMs) have shown significant potential in design-to-code tasks, they often fail to accurately preserve the layout during code generation. To this end, we draw inspiration from the *Chain-of-Thought* (CoT) reasoning in human cognition and propose LaTCoder, a novel approach that enhances layout preservation in webpage design during code generation with *Layout-as-Thought* (LaT). Specifically, we first introduce a simple yet efficient algorithm to divide the webpage design into image blocks. Next, we prompt MLLMs using a CoT-based approach to generate code for each block. Finally, we apply two assembly strategies—absolute positioning and an MLLM-based method—followed by dynamic selection to determine the optimal output. We evaluate the effectiveness of LaTCoder using multiple backbone MLLMs (i.e., DeepSeek-VL2, Gemini, and GPT-4o) on both a public benchmark and a newly introduced, more challenging benchmark (CC-HARD) that features complex layouts. The experimental results on automatic metrics demonstrate significant improvements. Specifically, TreeBLEU scores increased by 66.67% and MAE decreased by 38% when using DeepSeek-VL2, compared to direct prompting. Moreover, the human preference evaluation results indicate that annotators favor the webpages generated by LaTCoder in over 60% of cases, providing strong evidence of the effectiveness of our method.

CCS Concepts

• **Software and its engineering → Source code generation**.

Keywords

UI Automation; Code Generation; Design to Code

ACM Reference Format:
Yi Gui, Zhen Li, Zhongyi Zhang, Guohao Wang, Tianpeng Lv, Gaoyang Jiang, Yi Liu, Dongping Chen, Yao Wan, Hongyu Zhang, Wenbin Jiang, Xuanhua Shi, and Hai Jin. 2025. LaTCoder: Converting Webpage Design to Code with Layout-as-Thought. In *Proceedings of the 31st ACM SIGKDD*

*Also with National Engineering Research Center for Big Data Technology and System, Services Computing Technology and System Lab, Cluster and Grid Computing Lab, School of Computer Science and Technology, Huazhong University of Science and Technology, Wuhan, 430074, China.
†These authors contribute equally to this research.
‡Yao Wan is the corresponding author (wanyao@hust.edu.cn).

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
KDD '25, Toronto, ON, Canada
© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-1454-2/2025/08
https://doi.org/10.1145/3711896.3737016

Conference on Knowledge Discovery and Data Mining V.2 (KDD '25), August 3–7, 2025, Toronto, ON, Canada. ACM, New York, NY, USA, 12 pages. https://doi.org/10.1145/3711896.3737016

KDD Availability Link:
The source code of this paper has been made publicly available at https://doi.org/10.5281/zenodo.15515215.

1 Introduction

Front-end developers typically write webpage code based on *Graphical User Interface* (GUI) mockups created by UI designers. This process involves translating visual components—such as elements, layouts, and functionalities—into *Hypertext Markup Language* (HTML), *Cascading Style Sheets* (CSS), and JavaScript code, which is often time-consuming and costly. Due to the significant burden of generating large amounts of repetitive webpage code, more than 75.8% of front-end developers have adopted AI tools to improve development efficiency[1]. Consequently, there is an increasing need for automated design-to-code solutions that can transform a webpage design into code.

Previously, several efforts aimed to generate UI code from simple-styled design images (such as hand-drawn sketches [37]) using smaller models [7]. Recently, the powerful capabilities of *Multimodal Large Language Models* (MLLMs), such as Pix2Struct [27], GPT-4V [34], and Claude [4], have made it possible to directly convert high-resolution webpage designs into code. Several works have curated large-scale corpora for training purposes, including WebSight [26], WebCode2M [16], and Web2Code [54]. Notably, Si et al. [42] established a high-quality benchmark and introduced a novel metric (i.e., visual score) specifically tailored for evaluating the performance of MLLMs. Building on these foundational efforts, subsequent studies have focused on fine-tuning task-specific MLLMs [16, 26] or instructing interactive MLLMs with augmented methods such as self-revision [42]. In addition, Wan et al. [47] proposed DCGen, which improves code generation by synthesizing code directly from the entire design while incorporating natural language descriptions for subregions.

Despite the promising performance achieved by previous studies, we are still far from fully automating UI synthesis for real-world webpages. Existing methods primarily rely on *monolithic* generation, where the complete webpage code is generated directly from the design using MLLMs. However, we observe that this approach is inherently limited, as *partial layout information is often lost during code generation.* Consequently, MLLMs struggle to preserve the original structure and layout when dealing with real-world webpages that contain diverse styles and complex content.

For better illustration, Figure 1 presents a real-world example (with minor modifications) from the well-known project *screen-to-shot* [2], where MLLMs generate corresponding code from website screenshots using meticulously crafted prompts. Figure 1(a) displays a screenshot from an Instagram page, while Figure 1(b) shows the corresponding webpage synthesized by GPT-4V. As observed, the layout of the synthesized webpage differs significantly from the original design, particularly in the elements highlighted in boxes. GPT-4V incorrectly arranges the elements vertically instead of horizontally, even when instructed to "*Make sure to always get the layout right*". Our further investigation reveals that this limitation of easily losing partial layout information when translating designs into webpage code is not an exception but a limitation shared by many MLLMs. We speculate that this limitation may be due to the vulnerabilities [6] of MLLMs in factual interpretation [8] and numerical reasoning [21], which affects accurate capture of element positions and sizes in webpage designs.

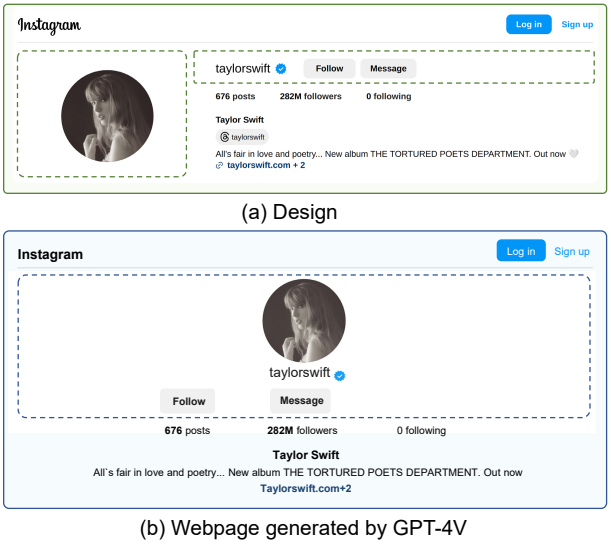


Figure 1: A real-world bad case from the famous project, *screenshot-to-code* [2], where GPT-4V incorrectly arranges the elements during generation (as highlighted in boxes).

Our Work. To mitigate the limitation of MLLMs in accurately capturing layout information, we propose a novel approach called LaTCoder. Our method incorporates an efficient divider, a CoT-based code generator, and a flexible code assembler. Drawing inspiration from the success of *Chain-of-Thought* (CoT), which decomposes complex tasks into simpler steps solvable by LLMs in sequence, we introduce a similar concept: *Layout-as-Thought* (LaT), which generates webpage code block by block, as opposed to traditional monolithic generation. In this approach, the design is broken down into a series of image blocks, each regarded as a "thought" that can be processed independently. We begin by dividing the webpage design into distinct image blocks in subregions, represented as bounding boxes (BBoxes). Using these BBoxes, the design is cropped into image blocks, which are then input into MLLMs one at a time for subregion code generation via CoT-based prompts. Next, guided by the BBoxes, we assemble the generated code for each image block using both absolute positioning and MLLM-based assembly to form the complete webpage code. Finally, we introduce a verifier to validate the results from both assembly strategies, further enhancing performance. Since each block is anchored by its BBox within the overall layout, LaTCoder significantly alleviates the limitation of MLLMs in capturing layout information accurately. Furthermore, by breaking the monolithic generation process into block-by-block code generation, LaTCoder reduces the burden on

[1]https://tsh.io/state-of-frontend/

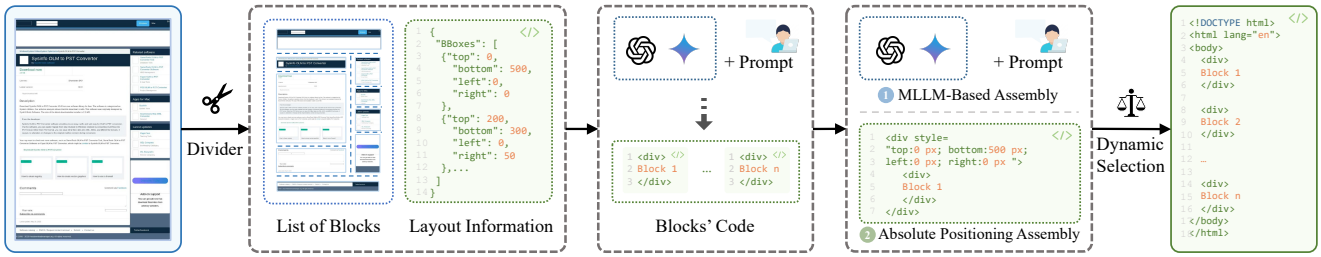


Figure 2: The workflow of LaTCoder.

MLLMs to generate lengthy code, resulting in improved accuracy in detail generation.

To validate the effectiveness and generalizability of LaTCoder, we introduce a more challenging dataset, featuring more complex layouts. Specifically, we manually sample from the Common Crawl dataset [1] and generate paired data to curate the dataset, which we refer to as CC-HARD. We evaluate our method and four state-of-the-art baseline approaches using different backbone MLLMs on a public dataset (i.e., Design2Code-Hard) and our newly introduced CC-HARD. Experimental results show that integrating various MLLMs as backbones into our method significantly boosts performance across all automatic metrics, particularly on Tree-BLEU [16], which measures the similarity of sub-structures in the HTML *Document Object Model* (DOM) tree. Moreover, we conduct a pairwise human preference evaluation of our method against each baseline, using majority voting from six annotators. The results of this evaluation demonstrate that, in most cases, human annotators prefer the webpages generated by our method, providing strong evidence for its effectiveness.

Contributions. The key contributions of this paper are as follows:

- We propose LaTCoder, a novel approach that enhances layout preservation in converting webpage designs to code using MLLMs with LaT.
- To assess the effectiveness of MLLMs, we introduce a new benchmark featuring more complex layouts, driving further advancements in MLLMs for webpage design-to-code generation.
- We conduct extensive experiments to validate the effectiveness of our approach, evaluated on the Design2Code-HARD and CC-HARD datasets. All the materials are available at: https://github.com/CGCL-codes/naturalcc/tree/main/examples/latcoder.

2 Design-to-Code: The Problem

The design-to-code task aims to translate design images—such as screenshots of existing websites or design mockups created by designers—into corresponding code. A typical modern webpage consists of three core components: HTML, which defines structural elements (e.g., `<div>`, `<button>`, `<img>`) and their hierarchical relationships; CSS, which controls layout properties (e.g., position, flexbox, grid) and visual styling (e.g., margin, padding, font size) to determine spatial arrangement and rendering; and JavaScript, which implements functionalities such as event handling and dynamic content updates. Our work focuses on static webpage synthesis from designs, where the rendered appearance is jointly determined by the HTML structure and CSS styling. Consequently, webpages with the same HTML code can have entirely different appearances depending on their CSS. The key of layout preservation in the design-to-code task with MLLMs is to accurately capture and map the size and position information of elements in the design into HTML/CSS code.

Given a high-resolution webpage design, LaTCoder aims to automatically generate the corresponding HTML and CSS code. The resulting webpage, once rendered, should closely resemble the input design in terms of layout, styling, and content. The design-to-code task can be defined as a mapping function $F : I \rightarrow (H, S)$, where $I \in \mathbb{R}^{M \times N \times 3}$ is the input webpage design with a size of $M \times N$, $H = \{h_1, h_2, \ldots, h_n\}$ represents the generated HTML elements, and $S = \{s_j \mid s_j = (p_j, v_j)\}$ specifies the CSS rules. Here, (p_j, v_j) denotes a pair of property and value in CSS. The objective is to minimize the visual difference between the rendered output $R = Render(H \cup S)$ and the input I:

$$\hat{H}, \hat{S} = \arg\min_{H,S} D(I, R),$$

where $\hat{H}$, $\hat{S}$ are the optimal HTML and CSS that minimize the perceptual distance $D(I, R)$ between the input design I and the rendered output R.

3 LaTCoder: Our Approach

Figure 2 illustrates the workflow of our proposed LaTCoder, which is composed of three components: (a) layout-aware division, (b) block-wise code synthesis, and (c) layout-preserved assembly. We first divide the design into smaller image blocks using an algorithm that ensures text integrity while recording the corresponding BBox information. Furthermore, we instruct MLLMs to generate code for each block using CoT-based prompts. Finally, we assemble the generated block code using both absolute positioning and MLLM-based strategies, followed by dynamic selection to get the best output. By anchoring blocks to their original positions in the design, this approach maximizes layout preservation while significantly reducing the burden on MLLMs when generating lengthy code.

3.1 Layout-Aware Division

Dividing the webpage design into appropriately sized image blocks is an essential step for generating code incrementally. Traditional image segmentation methods like Mask R-CNN [20] and U-Net [39] detect irregular object boundaries through instance or semantic segmentation [41]. In contrast, the vast majority of webpage layouts follow CSS box model conventions (as illustrated in Table 1), where structured, rectangular containers form the fundamental building

Table 1: Statistics of non-standard cases across three public datasets: IL refers to irregular layouts with misaligned or unevenly spaced elements; OL indicates overlapping layouts; GB stands for gradient backgrounds.

Dataset	Size	IL	OL	GB	Total
Design2Code	485	11	5	5	21 (4.33%)
Design2Code-HARD	80	5	1	5	11 (13.75%)
WebCode2M-Long	256	1	6	1	8 (3.13%)

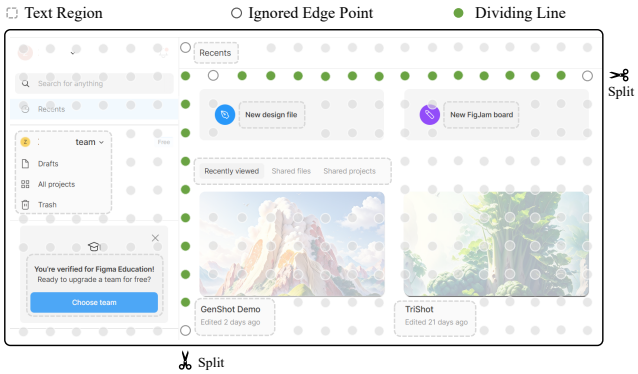


Figure 3: A toy example of dividing line detection.

blocks. Therefore, we propose a specialized and efficient algorithm for detecting horizontal/vertical lines to divide design mockups into grid-aligned blocks, better aligning with the structured nature of HTML/CSS layout systems.

Dividing Line Detection. To divide the design image into appropriately sized and grid-aligned blocks, we aim to identify a set of horizontal/vertical lines that can fully partition the image. There are other ways [47] to define and search for such dividing lines, but our focus is on exploring the potential of LaT in webpage generation, rather than finding the optimal dividing algorithm. Therefore, we adopt a simple yet effective definition for the dividing lines: horizontal or vertical solid-colored lines, with the distance to the nearest adjacent dividing line no greater than a predefined threshold τ. To achieve this, we design a search algorithm (as shown in Algorithm 1), which scans line by line in only one direction (either horizontal or vertical) to find the set of dividing lines. The algorithm is applied recursively to the original design image and each sub-image until no further dividing lines can be found, ultimately gathering all dividing lines.

Algorithm Optimizations. In practice, we propose the following optimizations to enhance the accuracy and efficiency of the detection algorithm, as exemplified in Figure 3. **(1) Ensuring text region integrity.** Dividing lines may split text regions, particularly when they span across line or paragraph gaps. To avoid this, we integrate *Optical Character Recognition* (OCR) to detect text regions and ensure that a line is only considered a valid dividing line if it does not go through any text regions. **(2) Improving the efficiency via grid sampling.** Pixel-level scanning on high-resolution design images is inefficient and computationally expensive. Instead, we apply a grid sampling technique, where pixels on the image are

Algorithm 1 Get dividing lines in image I

Require: Image I, minimum distance threshold τ
Ensure: A set S of dividing lines

Initialize $h \leftarrow$ left edge, $v \leftarrow$ top edge
Initialize empty set S
for each row starting from h **do**
  Let $h1$ be the next candidate line
  if `IsSolidColored`$(h1)$ **and** `Distance`$(h, h1) \geq \tau$ **then**
    Add $h1$ to S
    Update $h \leftarrow h1$
  end if
end for
if $S \neq \emptyset$ **then**
  Return S
end if
for each column starting from v **do**
  Let $v1$ be the next candidate line
  if `IsSolidColored`$(v1)$ **and** `Distance`$(v, v1) \geq \tau$ **then**
    Add $v1$ to S
    Update $v \leftarrow v1$
  end if
end for
Return S

sampled at fixed intervals when determining a solid-colored line, rather than scanning at the step of every individual pixel. **(3) Ignoring a few points at edges.** Pixels in the borders of images may hinder the determination of a solid-colored line. To mitigate this, we ignore the first few pixels at the edges during the determination.

With all dividing lines detected, the design is divided into a list of blocks of subregions, represented as BBoxes. Blocks that are smaller than a predefined threshold θ will be merged into adjacent blocks. These BBoxes are recorded further for block-wise code generation and assembly. More details can be found in Section 4.4.

A Toy Example. Figure 3 displays a toy example of detecting dividing lines. First, we sample points at fixed intervals across the entire image, forming a grid. Next, we scan the grid row by row or column by column to identify potential dividing lines. During this process, a valid dividing line must be a solid-colored line that does not cross any text regions. To minimize the impact of borders, we ignore one edge pixel in the determination. As a result, the algorithm detects one vertical and one horizontal segmentation line, marked by green points in the figure. Notably, although the edge pixel is ignored, it is still considered part of the dividing line.

3.2 Block-Wise Code Synthesis

The goal of this module is to generate code snippets for image blocks. We first crop out all image blocks from the design using their BBoxes, and then input each image block into interactive MLLMs one by one for code generation. We have meticulously designed a prompt for generating accurate and high-quality webpage code. Our prompt (see Figure 7 in Appendix) design mainly adheres to the following principles: **(1) Use a unified webpage template.** Generate Tailwind-style HTML/CSS code for a `div` within a fixed webpage template. This unified template ensures that the styles

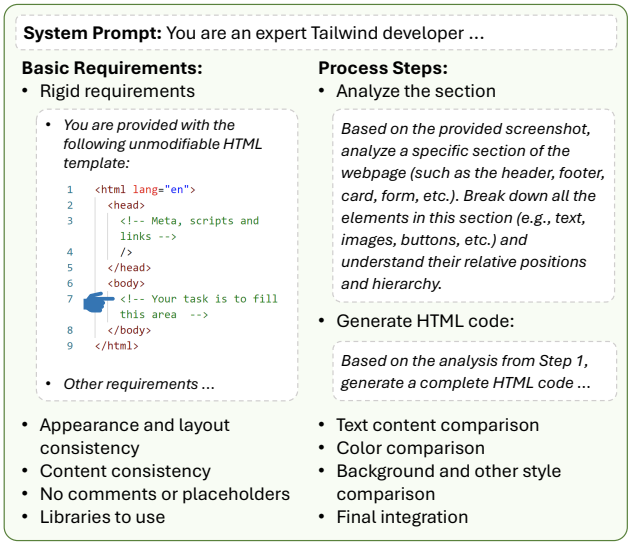


Figure 4: The simplified prompt for generating image block code (the full version is shown in Figure 7 in Appendix).

of all blocks' code remain consistent after assembling. **(2) Layout first.** Focus first on the appearance and layout consistency, then strive to maintain content consistency. **(3) CoT-based generation.** Generate step by step: analyze the image block, generate initial HTML/CSS code, check the code on text content, color, background, and other styles, then polish the code to finalize the generation. For weaker models, such as DeepSeek-VL2, due to the shorter context limit, we provide a slightly simplified version of the prompt, which can be found in the artifacts. For better illustration, we provide a simplified version of the complete prompt in Figure 4.

3.3 Layout-Preserved Assembly

We explore two distinct strategies for assembling the code of all image blocks into complete code while preserving the overall design layout: absolute positioning and MLLM-based assembly. MLLM-based assembly is more flexible but requires that MLLMs support a longer context to handle the code of all image blocks effectively. In contrast, absolute positioning assembly is faster and particularly suitable for weaker MLLMs with shorter context windows. Each strategy offers unique advantages—absolute positioning excels in position accuracy, while MLLM-based assembly often produces more aesthetically pleasing results. As a result, we retain both strategies where applicable and introduce dynamic selection to get the best output.

Strategy 1: Absolute Positioning Assembly (APS). We utilize the coordinates in the BBoxes obtained during the division stage to perform absolute positioning assembly. Each image block's code is encapsulated within a parent `<div>`, with its position and size set according to the BBox.

Strategy 2: MLLM-Based Assembly (MS). For interactive MLLMs, constraints can be applied by adjusting the prompt to generate more accurate and high-quality webpage code. Therefore, we design the prompt (as shown in the artifacts) following the main principles outlined below to guide the code assembly: **(1) Preserve the layout:** merge the provided blocks' code based on the original design image and the BBox information of each image block, ensuring that the positions match those in the design. **(2) CoT-based assembly:** analyze the layout information using all BBoxes, assemble the code, compare the generated webpage with the design image to ensure content, layout, and style consistency—fixing discrepancies if needed—then refine and finalize the complete code.

Dynamic Strategy Selection. The dynamic strategy selection aims to determine the best output when both assembly strategies are available. If the MLLM has a sufficiently long context window to process all image block codes, both strategies can be considered. However, when using a weaker MLLM, such as DeepSeek-VL2, only absolute positioning is employed due to its limited context capacity. We design a verifier to evaluate the similarity between the generated webpage and the webpage design. Importantly, this evaluation is **reference-free**, meaning it does not rely on the source code of the target. Consequently, we compare the screenshots of the generated webpage and the original design, providing a practical evaluation mechanism. Inspired by [54], we explore the potential of employing the *MLLM-as-a-Judge* paradigm for implementing the verifier. Although *MLLM-as-a-Judge* has achieved notable success across multiple domains [12–14, 35], we find it is not capable of reliably and accurately verifying the similarity or quality of images in our scenario, even with meticulously designed prompts (see them in artifacts). As a result, we return to traditional automatic metrics for assessing image similarity. Previous studies [45] suggest that Mean Absolute Error (**MAE**) and Normalized Earth Mover's Distance (**NEMD**) [40] align more closely with human preference on the webpage similarity. However, since both metrics operate at the pixel level, they may overlook semantic (high-level) information. To mitigate this limitation, we combine **MAE** and **CLIP** [36] similarity into a composite metric for the verifier. The metric, called the verify score, is defined as follows:

$$\text{Verify Score} = \frac{1}{2} \times \left(1 - \frac{\text{MAE}}{255}\right) + \frac{1}{2} \times \text{CLIP}\,.$$

We empirically set equal coefficients (0.5) for the two components in this formula.

4 Experimental Setup

4.1 Evaluation Datasets

Design2Code-HARD. One of the primary benchmarks for webpage generation is Design2Code [42]. However, according to the latest version of its paper, when using stronger models, such as GPT-4o, the performance gap between direct generation and various augmented methods on the Design2Code dataset has become minimal. As a result, we have adopted its newly proposed, more complex version—Design2Code-HARD—as one of our text benchmarks. This version includes 80 extremely long samples.

CC-HARD: A More Challenging Benchmark. However, during testing, we found that the complexity of Design2Code-HARD mainly lies in the text length, while its layout and structure remain relatively simple. As a result, **the performance differences between various methods are still minimal on Design2Code-HARD**. To address this, we instruct two experts to manually obtain

Table 2: A statistical comparison between Design2Code-HARD and CC-HARD.

	Design2Code-HARD	CC-HARD
Size	80	128
Avg. Len (tokens)	8900±2399	8416±2190
Avg. Text Len (tokens)	3554±2820	969±762
Avg. Tags	251±232	274±66
Avg. DOM Depth	10±4	16±3
Avg. Unique Tags	23±5	27±5

more challenging samples from the Common Crawl [1] dataset and generate paired data, curating a new benchmark, called CC-HARD. We conduct a statistical analysis of the two benchmarks, as shown in Table 2. While the overall average length distributions of the two datasets are similar, the textual content in Design2Code-HARD is much longer than in CC-HARD. This suggests that the total length of CC-HARD is consumed more by source code rather than textual content. Compared to Design2Code-HARD, CC-HARD contains significantly more tags and unique tags, as well as a deeper DOM tree. These differences indicate that CC-HARD is more challenging in terms of layout and structure, which is further supported by our experimental results.

4.2 Evaluation Metrics

We evaluate the generated samples using various automatic metrics in terms of **code and visual similarity** and select the following four metrics for presentation in the main text, which are more representative, relevant, and aligned with human preferences:

- **TreeBLEU** [16]. This metric measures the structural similarity of the HTML DOM tree by calculating the recall of 1-height subtrees, relative to the reference.
- **CLIP** [36]. This metric evaluates content similarity by computing the CLIP cosine similarity between the screenshot of the generated page and the original design.
- **Visual Score** [42][2]. A hybrid metric that calculates the match ratio of blocks and also considers block-level similarities in color, text, CLIP, and position.
- **MAE** (**M**ean **A**bsolute **E**rror). MAE measures the average absolute pixel color value difference between images.

4.3 Baselines

Backbone MLLMs. We use an open-source model and two commercial models as backbone MLLMs:

- **DeepSeek-VL2** [30]. An open-source and advanced series of large Mixture-of-Experts Vision-Language Models which has three variants: DeepSeek-VL2-tiny, DeepSeek-VL2-small, and DeepSeek-VL2, with 1.0B, 2.8B and 4.5B activated parameters respectively.
- **Gemini** (`gemini-v1.5-pro-latest`) [3]. DeepMind's Gemini specializes in seamless cross-modal reasoning, natively integrating text, code, and visual modalities through unified architecture.
- **GPT-4o** (`v2024-02-01`) [32] OpenAI's GPT-4o prioritizes text coherence and task generalization, leveraging massive-scale pretraining to handle complex logical chains.

.

Comparison Methods. We include four state-of-the-art methods and two variants of LaTCoder as baseline methods:

- We include three baseline methods from Design2Code [42]: **direct**, **text-augmented**, and **self-revision**, using exactly the same prompts and settings.
- **DCGen** [47], with `max_depth=1` as specified in its paper.
- **LaTCoder (APS)** and **LaTCoder (MS)**. Since our method employs two distinct strategies—Absolute Positioning Assembly (APS) and MLLM-based Assembly (MS)—for assembling blocks' code, we treat these two variants as independent baselines for comparative experiments.

Given that [42] and [16] have demonstrated GPT-4o's superior performance over task-specified models such as WebSight-8B [26], Design2Code-18B [42], and WebCoder [16], we have excluded these models from our baseline comparisons.

4.4 Implementation Details

Common Settings. In order to improve the reproducibility of experimental results, we set the temperature parameter of all APIs to 0 and fixed the random seeds for Python's `random`, `torch`, and `numpy` packages to 2026. All the experiments are conducted on a Linux server equipped with 4 NVIDIA A800 80GB GPUs.

Detailed Settings for the Dividing Algorithm. While we introduce the dividing algorithm in Section 3, we have omitted many trivial details to help readers focus on the key points. We use EasyOCR[3] to obtain the BBoxes of all text areas and then merge adjacent BBoxes that are within 20 pixels horizontally or vertically, allowing for more accurate determination of complete text paragraphs. We set the grid sampling interval to 5 pixels, the minimum dividing distance (τ) to 50 pixels, and the number of ignored edge points to 10. Additionally, we limit the maximum depth of recursive searches to 3. When merging blocks, we set the allowed minimum block area (θ) to 300×300 pixels. These parameters ensure that the final division does not result in overly coarse or fine-grained blocks, achieving a relatively balanced visual outcome.

5 Experimental Results and Analysis

5.1 Overall Performance

Effectiveness of LaTCoder. As depicted in Table 3, compared to other methods, LaTCoder with GPT-4o shows improvements of 17.65% in TreeBLEU, 1.27% in CLIP, 3.8% in Visual Score, and a 37.41% reduction in MAE on Design2Code-HARD; on CC-HARD, it improves by 60% in TreeBLEU, 2.53% in CLIP, 2.56% in Visual Score, and reduces MAE by 43.23%. Similarly, with Gemini, LaTCoder shows improvements of 2.38% in CLIP, 5.19% in Visual Score, and a 40.09% reduction in MAE on Design2Code-HARD, and improvements of 30% in TreeBLEU, 2.56% in CLIP, 2.63% in Visual Score, and a 43.03% reduction in MAE on CC-HARD. With DeepSeek-VL2, LaTCoder shows significant improvements in TreeBLEU (58.33%),

[2]We use the implementation of visual score from the first version of their paper, which may differ from the latest version, particularly regarding the sub-indicator *text color*

[3]https://github.com/JaidedAI/EasyOCR

Table 3: The overall performance of LaTCoder and baseline models with different backbone MLLMs across two datasets.

Method	Design2Code-HARD				CC-HARD			
	TreeBLEU(↑)	CLIP(↑)	Visual Score(↑)	MAE(↓)	TreeBLEU(↑)	CLIP(↑)	Visual Score(↑)	MAE(↓)
DeepSeek-VL2								
Direct Prompting	0.12(±0.07)	**0.81(±0.08)**	0.64(±0.25)	69.13(±28.53)	0.09(±0.04)	**0.75(±0.10)**	0.64(±0.30)	66.91(±21.30)
LaTCoder (APS)	**0.19(±0.09)**	0.77(±0.09)	**0.72(±0.19)**	**51.63(±36.89)**	**0.15(±0.05)**	0.74(±0.11)	**0.72(±0.22)**	**41.13(±24.41)**
Δ	58.33%	-4.94%	12.5%	-25.31%	66.67%	-1.33%	12.5%	-38.53%
Gemini								
Direct Prompting	0.16(±0.09)	0.84(±0.08)	0.76(±0.19)	65.52(±31.45)	0.09(±0.04)	0.78(±0.10)	0.76(±0.23)	65.22(±24.68)
Text-Augmented	0.14(±0.09)	0.83(±0.09)	0.76(±0.17)	63.34(±30.52)	0.09(±0.05)	0.78(±0.09)	0.74(±0.24)	66.02(±24.00)
Self-Revision	0.15(±0.09)	0.84(±0.08)	0.77(±0.17)	62.59(±30.50)	0.10(±0.05)	0.77(±0.10)	0.75(±0.23)	67.20(±23.70)
DCGen	0.14(±0.09)	0.79(±0.09)	0.74(±0.17)	78.45(±29.94)	0.09(±0.05)	0.73(±0.12)	0.74(±0.24)	75.79(±25.79)
LaTCoder (MS)	**0.16(±0.07)**	0.83(±0.08)	**0.81(±0.07)**	59.72(±26.21)	0.13(±0.05)	0.78(±0.10)	0.76(±0.25)	58.52(±21.66)
LaTCoder (APS)	0.16(±0.08)	0.86(±0.07)	0.78(±0.17)	40.21(±25.02)	0.13(±0.05)	0.80(±0.09)	0.76(±0.25)	37.59(±18.85)
LaTCoder	0.16(±0.08)	**0.86(±0.06)**	0.78(±0.17)	**37.50(±21.50)**	**0.13(±0.04)**	**0.80(±0.09)**	**0.78(±0.23)**	**37.15(±17.52)**
Δ	0%	2.38%	5.19%	-40.09%	30%	2.56%	2.63%	-43.03%
GPT-4o								
Direct Prompting	0.16(±0.11)	0.84(±0.08)	0.75(±0.19)	61.62(±25.06)	0.09(±0.05)	0.79(±0.10)	0.76(±0.24)	66.18(±21.94)
Text-Augmented	0.16(±0.11)	0.86(±0.07)	0.79(±0.17)	54.21(±22.94)	0.10(±0.05)	0.79(±0.11)	0.78(±0.23)	64.88(±21.09)
Self-Revision	0.16(±0.10)	0.86(±0.07)	0.79(±0.17)	54.53(±24.23)	0.10(±0.05)	0.79(±0.10)	0.78(±0.23)	64.82(±21.11)
DCGen	0.17(±0.10)	0.83(±0.10)	0.77(±0.16)	63.31(±27.21)	0.10(±0.05)	0.74(±0.12)	0.75(±0.26)	68.31(±21.58)
LaTCoder (MS)	0.20(±0.11)	0.86(±0.07)	**0.82(±0.12)**	59.55(±25.84)	0.16(±0.06)	0.78(±0.09)	0.78(±0.25)	58.29(±20.45)
LaTCoder (APS)	0.20(±0.11)	0.86(±0.08)	0.80(±0.15)	36.21(±22.36)	0.16(±0.06)	0.80(±0.09)	0.80(±0.23)	37.53(±20.66)
LaTCoder	**0.20(±0.11)**	**0.87(±0.07)**	0.80(±0.15)	**33.93(±18.15)**	**0.16(±0.06)**	**0.81(±0.09)**	**0.80(±0.23)**	**36.80(±17.49)**
Δ	17.65%	1.27%	3.8%	-37.41%	60%	2.53%	2.56%	-43.23%

* **(1)** For DeepSeek-VL2, due to its limited context window, only two methods were tested, with absolute positioning applied during assembly.
(2) The three variants of LaTCoder correspond to different assembly strategies: using MLLMs, absolute positioning, and getting the best of the first two strategies with a dynamic verifier.
(3) Δ represents the improvement or decline of LaTCoder's best performance among three variances relative to the best of baselines.

Visual Score (12.5%), and MAE (reduced by 25.31%) on Design2Code-HARD, despite a slight drop in CLIP (-4.94%); on CC-HARD, it improves TreeBLEU by 66.67%, Visual Score by 12.5%, and reduces MAE by 38.53%, while CLIP decreases by -1.33%. These results demonstrate that LaTCoder outperforms nearly all baseline methods across all backbone MLLMs on both test benchmarks. We can conclude that **LaTCoder significantly boosts MLLMs' performance in webpage generation, especially for weaker models**.

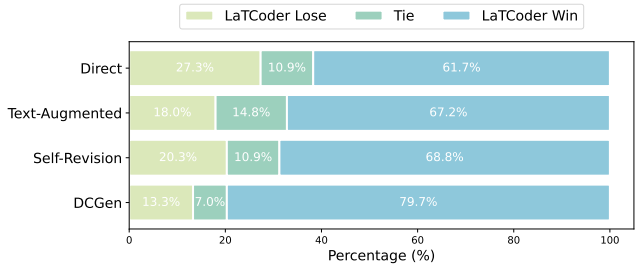


Figure 5: Pairwise human preference evaluation of baseline methods relative to LaTCoder, using GPT-4o as the backbone on the CC-HARD dataset, with majority voting from six annotators.

CC-HARD is More Challenging. For all three backbone models in Table 3, both LaTCoder and the baseline methods show performance drops on CC-HARD, especially in TreeBLEU and MAE. For example, when using GPT-4o, on CC-HARD compared to Design2Code-HARD, the average TreeBLEU, CLIP, and Visual Score of all methods decreased by 48.97%, 8.03%, and 1.24%, respectively, while MAE increased by 9.12%. The results suggest that **the complexity of CC-HARD is more challenging for all backbones and methods**. We attribute the challenge of CC-HARD to its more complex layouts, as shown in Table 2, with deeper DOM trees and a greater variety and number of tags.

5.2 Ablation Studies

Influence of Assembly Strategies. As shown in Table 3, when using Gemini and GPT-4o as backbones on both datasets, the performance differences between LaTCoder (MS) and LaTCoder (APS) in TreeBLEU, CLIP, and Visual Score are minimal, with both outperforming all baselines. This suggests that the assembly strategies have little impact on the code structure, content similarity, and block-level similarity of the generated webpage. However, LaTCoder (APS) significantly outperforms LaTCoder (MS) in MAE, likely because APS strictly preserves block positions, while MS may introduce positional changes. Using absolute positioning ensures

Table 4: Ablation study on the CoT-based generation prompt.

	TreeBLEU	CLIP	Visual Score	MAE
CoT-based Prompt	0.16	0.81	0.80	36.80
Simplified Prompt	0.13	0.76	0.71	43.17

Table 5: Performances under different model scales.

	TreeBLEU	CLIP	Visual Score	MAE
DeepSeek-VL2-tiny (3.37B, 1B activated)				
Direct	0.04	0.66	0.24	76.55
LaTCoder	0.11	0.73	0.67	44.12
Δ	+175%	+10.61%	+179.17%	-42.36%
DeepSeek-VL2-small (16.1B, 2.8B activated)				
Direct	0.08	0.73	0.61	61.83
LaTCoder	0.12	0.73	0.68	49.44
Δ	+50%	+0%	+11.48%	-20.04%
DeepSeek-VL2 (27.5B, 4.5B activated)				
Direct	0.09	0.75	0.64	66.91
LaTCoder	0.15	0.74	0.72	41.13
Δ	+66.67%	-1.33%	+12.5%	-38.53%

that blocks with similar content align more precisely, reducing MAE compared to the original design. However, we find that the results generated by MLLM-based assembly often have better overall aesthetics and smoother transitions between blocks compared to APS, which is another reason for retaining both assembly strategies. As shown in Table 3, filtering the results through a verifier further improves the generated outcomes.

Influence of CoT-based Prompts. To assess the effectiveness of the CoT-based prompt for block-wise code generation, we conduct an ablation study using a simplified prompt on the CC-HARD dataset, with GPT-4o as the backbone MLLM. The simplified prompt omits any CoT-based reasoning and is as follows: *you are a frontend developer, and your task is to convert a webpage screenshot into HTML and CSS code. Return format: '"html code"'*. As shown in Table 4, the performance drops significantly when using the simplified prompt, highlighting the effectiveness of the CoT-based prompt.

Influence of Model Scales. We also conduct a study on the performance of LaTCoder across different model scales on the CC-HARD dataset. In this study, we use three variants of DeepSeek-VL2 as backbone MLLMs: DeepSeek-VL2-tiny, DeepSeek-VL2-small, and DeepSeek-VL2. The results in Table 5 show that LaTCoder consistently achieves significant improvements, with particularly notable gains for smaller models, underscoring its general applicability.

Parameters for the Dividing Algorithm. Since the dividing algorithm plays a crucial role and relies on several parameters, we conduct a parameter study focusing on the most critical one: the minimum area threshold for block merging (θ). The experiments are conducted on the CC-HARD dataset using GPT-4o as the backbone MLLM. The results in Table 6 indicate that the threshold setting of `300*300` is close to optimal.

Table 6: Parameter study on the minimum area threshold for merging blocks in the dividing algorithm.

θ	TreeBLEU	CLIP	Visual Score	MAE
100*100	0.14	0.8	0.75	37.07
200*200	0.14	0.8	0.76	36.56
300*300	0.16	0.81	0.8	36.8
400*400	0.13	0.79	0.74	40.88
500*500	0.12	0.78	0.73	42.35

5.3 Human Evaluation

We perform a pairwise human preference evaluation of our method against all baseline methods, using GPT-4o as the backbone on the CC-HARD dataset. The annotators are asked, "*Which is the one that is more similar to the design image and of higher quality*?" when presented with a pair of shuffled generated samples alongside the design image. To reduce the subjectivity of human evaluation, we apply a majority voting strategy to determine the preference for each generated sample. The possible outcomes are classified as `win`, `tie`, or `lose`. Results in Figure 5 show that compared to each baseline method, annotators preferred our method in at least 60% of the cases. Notably, when compared to DCGen, our method is favored in 79.7% of cases. **This comprehensive comparison provides strong evidence of the superiority of our approach**.

5.4 Qualitative Analysis

Good Case Study. In Figure 6, we present a case study on samples generated by LaTCoder and other baseline methods. It can be seen that the direct and self-revision methods exhibit very similar performance. By contrast, DCGen, which enhances monolithic generation with natural language descriptions of local regions, even shows some performance degradation. In comparison, LaTCoder significantly improves overall structural and detail similarity by leveraging the advantages of LaT and division-based generation. Interestingly, we find that the self-revision method produces results nearly identical to those of the text-augmented method. This is likely because the self-revision method refines outputs generated by the text-augmented method, and MLLMs generally have limited capabilities for refinement. Therefore, we omit the results from the text-augmented method for brevity.

Error Case Analysis. During the experiments, we have identified two main errors that contributed to poor results: (1) Layout misarrangement issue in the block code generation. MLLMs sometimes incorrectly center the content of a small block at the bottom, which deviates from its intended position in the design, resulting in discrepancies between the generated output and the original design. This highlights an inherent limitation of MLLMs in capturing precise layout information from images, even when dealing with relatively simple-styled designs. While LaTCoder significantly mitigates this issue at the overall layout level, it cannot entirely eliminate it when generating code for subregions. (2) MLLMs' 'laziness' issue. Gemini sometimes takes shortcuts during code assembly, omitting some blocks' code and resulting in missing regions in the generated output. We plan to explore strategies to mitigate these MLLMs' limitations in future work.



Figure 6: Case study of samples generated by LaTCoder and other baseline methods with GPT-4o as the backbone MLLM: LaTCoder significantly outperforms the others, particularly in preserving the layout of the design.

6 Related Work

UI Automation. Early researches, constrained by limited computational resources, primarily focus on generating UI code from simple-styled design images using smaller models. For instance, pix2code [7] leveraged LSTM and CNN architectures to produce domain-specific languages (DSLs), while Sketch2code [37] explored both deep learning-based and object detection-based methods for UI prototyping from hand-drawn mockups. With the advancement of MLLMs [3, 4, 32], recent studies have sought to integrate MLLMs into UI automation. Some efforts focus on curating specialized training datasets[16, 26] to enhance MLLMs' capabilities in UI generation, while others aim to establish benchmarks and evaluation metrics [19, 42, 51, 54] to systematically assess performance and drive further progress in this domain. Additionally, research [17, 47, 55] has explored novel approaches to improve the visual aesthetics and interactive functionalities of generated UIs. Despite these advancements, full UI automation remains a distant goal, particularly when dealing with real-world webpage designs that feature intricate layouts and extensive code.

Code Intelligence. Neural language models have advanced code intelligence [44], enabling key tasks such as code summarization [48, 49], code search [23, 46], and code generation [10, 33, 43]. The development of code-focused LLMs has progressed through several stages. Early models like InCoder [15] introduced capabilities for code infilling and synthesis, enabling more flexible code generation. Further advancements were made with models such as WizardCoder [31], which incorporated instruction tuning to better follow complex prompts. More recent models like Qwen-Coder [52] and DeepSeek-Coder [18] have focused on scaling model sizes and training data, aiming to improve performance across diverse coding tasks. While most prior work focused on general-purpose LLMs for code generation, this paper addresses a distinct problem: generating code directly from webpage designs. This task requires an understanding of UI structures, layout constraints, and code synthesis, setting it apart from conventional code generation.

Step Reasoning in LLMs. Various methods have been proposed to address complex problems, aiming to mitigate hallucination issues while enhancing models' reasoning capabilities. For instance, Yao et al. [53] and Besta et al. [9] introduced *Tree-of-Thought* (ToT) and *Graph-of-Thought* (GoT), respectively, building on the foundational *Chain-of-Thought* (CoT)[50] framework. Recently, OpenAI released a powerful commercial model, o1, which enhanced GPT's reasoning abilities by incorporating step-by-step thinking and verification processes [28]. In contrast to these approaches, we propose LaTCoder, a method specifically tailored for the design-to-code task with layout-as-thought. LaTCoder preserves layout information and alleviates the burden of generating lengthy code for MLLMs.

Layout Understanding and Generation. Prior work has explored layout understanding and generation across tasks like text-to-layout and document-to-layout. Text-to-layout synthesizes plausible arrangements of elements, with [29] using in-context learning for versatility and efficiency, and [11, 25] employing diffusion models [22, 38] for controllable generation. In document layout, Huang et al. [24] pre-trained multimodal Transformers with unified text and image masking, while Appalaraju et al. [5] introduced DocFormerV2, trained on novel unsupervised tasks. These works, however, do not directly address LaTCoder 's goal of converting images into code.

7 Conclusion

In this work, we draw inspiration from the CoT reasoning in human cognition and introduce LaTCoder, a novel approach that enhances layout preservation during webpage design-to-code generation through *Layout-as-Thought* (LaT). Specifically, we propose a simple yet effective algorithm that divides the webpage design into image blocks, instructs MLLMs to generate code for each block using CoT reasoning, and then assembles the code for all blocks using two distinct strategies with dynamic selection. To further evaluate the performance of MLLMs in the design-to-code task, we introduce a new and more challenging dataset, CC-HARD, featuring complex layouts. We assess our method on both a public benchmark and CC-HARD, with experimental results—on both automatic metrics and human evaluation—robustly demonstrating the effectiveness of our approach.

Acknowledgments

This work is supported by the Major Program (JD) of Hubei Province (Grant No. 2023BAA024).

References

[1] 2024. *The* Common Crawl *dataset.* https://data.commoncrawl.org/

[2] 2024. *The* screen-to-shot *project on the Github.* https://github.com/abi/screenshot-to-code/

[3] Rohan Anil, Sebastian Borgeaud, Yonghui Wu, Jean-Baptiste Alayrac, Jiahui Yu, Radu Soricut, Johan Schalkwyk, Andrew M. Dai, Anja Hauth, Katie Millican, David Silver, Slav Petrov, Melvin Johnson, Ioannis Antonoglou, Julian Schrittwieser, Amelia Glaese, Jilin Chen, Emily Pitler, Timothy P. Lillicrap, Angeliki Lazaridou, Orhan Firat, James Molloy, Michael Isard, Paul Ronald Barham, Tom Hennigan, Benjamin Lee, Fabio Viola, Malcolm Reynolds, Yuanzhong Xu, Ryan Doherty, Eli Collins, Clemens Meyer, Eliza Rutherford, Erica Moreira, Kareem Ayoub, Megha Goel, George Tucker, Enrique Piqueras, Maxim Krikun, Iain Barr, Nikolay Savinov, Ivo Danihelka, Becca Roelofs, Anaïs White, Anders Andreassen, Tamara von Glehn, Lakshman Yagati, Mehran Kazemi, Lucas Gonzalez, Misha Khalman, Jakub Sygnowski, and et al. 2023. Gemini: A Family of Highly Capable Multimodal Models. *ArXiv* abs/2312.11805 (2023).

[4] Anthropic. 2024. The Claude 3 Model Family: Opus, Sonnet, Haiku. https://api.semanticscholar.org/CorpusID:268232499.

[5] Srikar Appalaraju, Peng Tang, Qi Dong, Nishant Sankaran, Yichu Zhou, and R. Manmatha. 2023. DocFormerv2: Local Features for Document Understanding. In *Proceedings of the AAAI Conference on Artificial Intelligence.*

[6] Zechen Bai, Pichao Wang, Tianjun Xiao, Tong He, Zongbo Han, Zheng Zhang, and Mike Zheng Shou. 2024. Hallucination of Multimodal Large Language Models: A Survey. *ArXiv* abs/2404.18930 (2024).

[7] Tony Beltramelli. 2018. pix2code: Generating Code from a Graphical User Interface Screenshot. In *Proceedings of the ACM SIGCHI Symposium on Engineering Interactive Computing Systems, EICS 2018, Paris, France, June 19-22, 2018.* ACM, 3:1–3:6.

[8] Emily M. Bender, Timnit Gebru, Angelina McMillan-Major, and Shmargaret Shmitchell. 2021. On the Dangers of Stochastic Parrots: Can Language Models Be Too Big?. In *Proceedings of the 2021 ACM Conference on Fairness, Accountability, and Transparency.*

[9] Maciej Besta, Nils Blach, Ales Kubicek, Robert Gerstenberger, Michal Podstawski, Lukas Gianinazzi, Joanna Gajda, Tomasz Lehmann, Hubert Niewiadomski, Piotr Nyczyk, and Torsten Hoefler. 2024. Graph of Thoughts: Solving Elaborate Problems with Large Language Models. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 38. 17682–17690.

[10] Zhangqian Bi, Yao Wan, Zheng Wang, Hongyu Zhang, Batu Guan, Fangxin Lu, Zili Zhang, Yulei Sui, Hai Jin, and Xuanhua Shi. 2024. Iterative Refinement of Project-Level Code Context for Precise Code Generation with Compiler Feedback. In *Proceedings of the Findings of the Association for Computational Linguistics, ACL 2024, Bangkok, Thailand and virtual meeting, August 11-16, 2024.* 2336–2353.

[11] Shang Chai, Liansheng Zhuang, and Fengying Yan. 2023. LayoutDM: Transformer-based Diffusion Model for Layout Generation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR).* 18349–18358.

[12] Dongping Chen, Ruoxi Chen, Shu Pu, Zhaoyi Liu, Yanru Wu, Caixi Chen, Benlin Liu, Yue Huang, Yao Wan, Pan Zhou, et al. 2024. Interleaved Scene Graph for Interleaved Text-and-Image Generation Assessment. In *Proceedings of the Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024.* OpenReview.net.

[13] Dongping Chen, Ruoxi Chen, Shilin Zhang, Yinuo Liu, Yaochen Wang, Huichi Zhou, Qihui Zhang, Pan Zhou, Yao Wan, and Lichao Sun. 2024. MLLM-as-a-Judge: Assessing Multimodal LLM-as-a-Judge with Vision-Language Benchmark. In *Proceedings of the International Conference on Machine Learning.*

[14] Tu Anh Dinh, Carlos Mullov, Leonard Barmann, Zhaolin Li, Danni Liu, Simon Reiß, Jueun Lee, Nathan Lerzer, Fabian Ternava, Jianfeng Gao, Alexander Waibel, Tamim Asfour, Michael Beigl, Rainer Stiefelhagen, Carsten Dachsbacher, Klemens Bohm, and Jan Niehues. 2024. SciEx: Benchmarking Large Language Models on Scientific Exams with Human Expert Grading and Automatic Grading. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing.*

[15] Daniel Fried, Armen Aghajanyan, Jessy Lin, Sida Wang, Eric Wallace, Freda Shi, Ruiqi Zhong, Wen-tau Yih, Luke Zettlemoyer, and Mike Lewis. 2022. Incoder: A generative model for code infilling and synthesis. *ArXiv* abs/2204.05999 (2022).

[16] Yi Gui, Zhen Li, Yao Wan, Yemin Shi, Hongyu Zhang, Bohua Chen, Yi Su, Dongping Chen, Siyuan Wu, Xing Zhou, Wenbin Jiang, Hai Jin, and Xiangliang Zhang. 2025. WebCode2M: A Real-World Dataset for Code Generation from Webpage Designs. In *Proceedings of the International World Wide Web Conference, WWW 2025, Sydney, April 28–May 2, 2024.*

[17] Yi Gui, Yao Wan, Zhen Li, Zhongyi Zhang, Dongping Chen, Hongyu Zhang, Yi Su, Bohua Chen, Xing Zhou, Wenbin Jiang, and Xiangliang Zhang. 2025. UICopilot: Automating UI Synthesis via Hierarchical Code Generation from Webpage Designs. In *Proceedings of the International World Wide Web Conference, WWW 2025, Sydney, April 28–May 2, 2024.*

[18] Daya Guo, Qihao Zhu, Dejian Yang, Zhenda Xie, Kai Dong, Wentao Zhang, Guanting Chen, Xiao Bi, Yu Wu, YK Li, et al. 2024. DeepSeek-Coder: When the Large Language Model Meets Programming–The Rise of Code Intelligence. *ArXiv* abs/2401.14196 (2024).

[19] Hongcheng Guo, Wei Zhang, Junhao Chen, Yaonan Gu, Jian Yang, Junjia Du, Binyuan Hui, Tianyu Liu, Jianxin Ma, Chang Zhou, and Zhoujun Li. 2024. IW-Bench: Evaluating Large Multimodal Models for Converting Image-to-Web. *ArXiv* abs/2409.18980 (2024).

[20] Kaiming He, Georgia Gkioxari, Piotr Dollár, and Ross B. Girshick. 2020. Mask R-CNN. *IEEE Trans. Pattern Anal. Mach. Intell.* 42, 2 (2020), 386–397.

[21] Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. 2021. Measuring Mathematical Problem Solving With the MATH Dataset. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks 1, NeurIPS Datasets and Benchmarks 2021, December 2021, virtual.*

[22] Jonathan Ho, Ajay Jain, and Pieter Abbeel. 2020. Denoising Diffusion Probabilistic Models. In *Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems 2020, NeurIPS 2020, December 6-12, 2020, virtual.*

[23] Fan Hu, Yanlin Wang, Lun Du, Xirong Li, Hongyu Zhang, Shi Han, and Dongmei Zhang. 2023. Revisiting Code Search in a Two-Stage Paradigm. In *Proceedings of the Sixteenth ACM International Conference on Web Search and Data Mining, WSDM 2023, Singapore, 27 February 2023 - 3 March 2023.* ACM, 994–1002.

[24] Yupan Huang, Tengchao Lv, Lei Cui, Yutong Lu, and Furu Wei. 2022. LayoutLMv3: Pre-training for Document AI with Unified Text and Image Masking. In *Proceedings of the 30th ACM International Conference on Multimedia.*

[25] Naoto Inoue, Kotaro Kikuchi, Edgar Simo-Serra, Mayu Otani, and Kota Yamaguchi. 2023. LayoutDM: Discrete Diffusion Model for Controllable Layout Generation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR).* 10167–10176.

[26] Hugo Laurençon, Léo Tronchon, and Victor Sanh. 2024. Unlocking the conversion of Web Screenshots into HTML Code with the WebSight Dataset. *ArXiv* abs/2403.09029 (2024).

[27] Kenton Lee, Mandar Joshi, Iulia Raluca Turc, Hexiang Hu, Fangyu Liu, Julian Martin Eisenschlos, Urvashi Khandelwal, Peter Shaw, Ming-Wei Chang, and Kristina Toutanova. 2023. Pix2Struct: Screenshot Parsing as Pretraining for Visual Language Understanding. In *Proceedings of the International Conference on Machine Learning, ICML 2023, 23-29 July 2023, Honolulu, Hawaii, USA*, Vol. 202. 18893–18912.

[28] Hunter Lightman, Vineet Kosaraju, Yuri Burda, Harrison Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. 2024. Let's Verify Step by Step. In *Proceedings of the Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024.* OpenReview.net.

[29] Jiawei Lin, Jiaqi Guo, Shizhao Sun, Zijiang Yang, Jian-Guang Lou, and Dongmei Zhang. 2023. LayoutPrompter: Awaken the Design Ability of Large Language Models. In *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023.*

[30] Haoyu Lu, Wen Liu, Bo Zhang, Bing-Li Wang, Kai Dong, Bo Liu (Benjamin Liu), Jingxiang Sun, Tongzheng Ren, Zhuoshu Li, Hao Yang, Yaofeng Sun, Chengqi Deng, Hanwei Xu, Zhenda Xie, and Chong Ruan. 2024. DeepSeek-VL: Towards Real-World Vision-Language Understanding. *ArXiv* abs/2403.05525 (2024).

[31] Ziyang Luo, Can Xu, Pu Zhao, Qingfeng Sun, Xiubo Geng, Wenxiang Hu, Chongyang Tao, Jing Ma, Qingwei Lin, and Daxin Jiang. 2023. WizardCoder: Empowering Code Large Language Models with Evol-Instruct. *ArXiv* abs/2306.08568 (2023).

[32] OpenAI. 2023. GPT-4 Technical Report. *ArXiv* abs/2303.08774 (2023).

[33] Geliang Ouyang, Jingyao Chen, Zhihe Nie, Yi Gui, Yao Wan, Hongyu Zhang, and Dongping Chen. 2025. nvAgent: Automated Data Visualization from Natural Language via Collaborative Agent Workflow. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics, ACL 2025, Vienna, Austria, June 27–August 1, 2025.*

[34] Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke E. Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul Francis Christiano, Jan Leike, and Ryan J. Lowe. 2022. Training language models to follow instructions with human feedback. *ArXiv* abs/2203.02155 (2022).

[35] Shu Pu, Yaochen Wang, Dongping Chen, Yuhang Chen, Guohao Wang, Qi Qin, Zhongyi Zhang, Zhiyuan Zhang, Zetong Zhou, Shuang Gong, et al. 2025. Judge Anything: MLLM as a Judge Across Any Modality. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2, KDD 2025, Toronto, August 3–7, 2025.*

[36] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. 2021. Learning Transferable Visual Models From Natural Language Supervision. In *Proceedings of the 38th International Conference on Machine Learning, ICML 2021, 18-24 July 2021, Virtual Event*, Vol. 139. PMLR, 8748–8763.

[37] Alex Robinson. 2019. Sketch2code: Generating a website from a paper mockup. *ArXiv* abs/1905.13750 (2019).

[38] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. 2022. High-Resolution Image Synthesis with Latent Diffusion Models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR 2022, New Orleans, LA, USA, June 18-24, 2022*. IEEE, 10674–10685.

[39] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. 2015. U-Net: Convolutional Networks for Biomedical Image Segmentation. In *Proceedings of the Medical Image Computing and Computer-Assisted Intervention - MICCAI 2015 - 18th International Conference Munich, Germany, October 5 - 9, 2015, Proceedings, Part III (Lecture Notes in Computer Science, Vol. 9351)*. Springer, 234–241.

[40] Yossi Rubner, Carlo Tomasi, and Leonidas J. Guibas. 2000. The Earth Mover's Distance as a Metric for Image Retrieval. *International Journal of Computer Vision* 40 (2000), 99–121.

[41] Evan Shelhamer, Jonathan Long, and Trevor Darrell. 2014. Fully convolutional networks for semantic segmentation. In *Proceedings of the 2015 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 3431–3440.

[42] Chenglei Si, Yanzhe Zhang, Ryan Li, Zhengyuan Yang, Ruibo Liu, and Diyi Yang. 2025. Design2Code: Benchmarking Multimodal Code Generation for Automated Front-End Engineering. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL 2025 - Volume 1: Long Papers, Albuquerque, New Mexico, USA, April 29 - May 4, 2025*. Association for Computational Linguistics, 3956–3974.

[43] Zhihong Sun, Yao Wan, Jia Li, Hongyu Zhang, Zhi Jin, Ge Li, and Chen Lyu. 2024. Sifting through the Chaff: On Utilizing Execution Feedback for Ranking the Generated Code Candidates. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering, ASE 2024, Sacramento, CA, USA, October 27 - November 1, 2024*. ACM, 229–241.

[44] Yao Wan, Zhangqian Bi, Yang He, Jianguo Zhang, Hongyu Zhang, Yulei Sui, Guandong Xu, Hai Jin, and Philip Yu. 2024. Deep learning for code intelligence: Survey, benchmark and toolkit. *Comput. Surveys* 56, 12 (2024), 1–41.

[45] Yuxuan Wan, Yi Dong, Jingyu Xiao, Yintong Huo, Wenxuan Wang, and Michael R. Lyu. 2024. MRWeb: An Exploration of Generating Multi-Page Resource-Aware Web Code from UI Designs. *ArXiv* abs/2412.15310 (2024).

[46] Yao Wan, Jingdong Shu, Yulei Sui, Guandong Xu, Zhou Zhao, Jian Wu, and Philip Yu. 2019. Multi-modal attention network learning for semantic source code retrieval. In *Proceedings of 34th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 13–25.

[47] Yuxuan Wan, Chaozheng Wang, Yi Dong, Wenxuan Wang, Shuqing Li, Yintong Huo, and Michael R. Lyu. 2024. Automatically Generating UI Code from Screenshot: A Divide-and-Conquer-Based Approach. *ArXiv* abs/2406.16386 (2024).

[48] Yao Wan, Zhou Zhao, Min Yang, Guandong Xu, Haochao Ying, Jian Wu, and Philip S Yu. 2018. Improving automatic source code summarization via deep reinforcement learning. In *Proceedings of the 33rd ACM/IEEE international conference on automated software engineering*. 397–407.

[49] Wenhua Wang, Yuqun Zhang, Yulei Sui, Yao Wan, Zhou Zhao, Jian Wu, S Yu Philip, and Guandong Xu. 2020. Reinforcement-learning-guided source code summarization using hierarchical attention. *IEEE Transactions on software Engineering* 48, 1 (2020), 102–119.

[50] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. 2022. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. In *Advances in Neural Information Processing Systems 35: Annual Conference on Neural Information Processing Systems 2022, NeurIPS 2022, New Orleans, LA, USA, November 28 - December 9, 2022*.

[51] Jingyu Xiao, Yuxuan Wan, Yintong Huo, Zhiyao Xu, and Michael R. Lyu. 2024. Interaction2Code: How Far Are We From Automatic Interactive Webpage Generation? *ArXiv* abs/2411.03292 (2024).

[52] Jin Xu, Zhifang Guo, Jinzheng He, Hangrui Hu, Ting He, Shuai Bai, Keqin Chen, Jialin Wang, Yang Fan, Kai Dang, Bin Zhang, Xiong Wang, Yunfei Chu, and Junyang Lin. 2025. Qwen2.5-Omni Technical Report. *ArXiv* abs/2503.20215 (2025).

[53] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Tom Griffiths, Yuan Cao, and Karthik Narasimhan. 2023. Tree of Thoughts: Deliberate Problem Solving with Large Language Models. In *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*.

[54] Sukmin Yun, Haokun Lin, Rusiru Thushara, Mohammad Qazim Bhat, Yongxin Wang, Zutao Jiang, Mingkai Deng, Jinhong Wang, Tianhua Tao, Junbo Li, Haonan Li, Preslav Nakov, Timothy Baldwin, Zhengzhong Liu, Eric P. Xing, Xiaodan Liang, and Zhiqiang Shen. 2024. Web2Code: A Large-scale Webpage-to-Code Dataset and Evaluation Framework for Multimodal LLMs. In *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024*.

[55] Ti Zhou, Yanjie Zhao, Xinyi Hou, Xiaoyu Sun, Kai Chen, and Haoyu Wang. 2024. Bridging Design and Development with Automated Declarative UI Code Generation. *ArXiv* abs/2409.11667 (2024).

Prompt for Block-wise Code Synthesis

You are an expert Tailwind developer.
Based on the reference screenshot of a specific section of a webpage (such as the header, footer, card, etc.) provided by the user, build a single-page app using Tailwind, HTML, and JS. Please follow the detailed requirements below to ensure the generated code is accurate:

Basic Requirements:

(1) **Rigid Requirements**

- You are provided with the following unmodifiable HTML framework:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <script src="https://cdn.tailwindcss.com"></script>
    <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/5.15.3/css/all.min.css">
</head>
<body>
    <!-- Your task is to fill this area -->
</body>
</html>
```

- Your task is to generate a code block that starts with a tag and ends with a tag, and embed it within the <body> tag of the above-mentioned framework.
- Do not deliberately center the content. Arrange the elements according to their original layout and positions.
- The generated code should not have fixed width and height settings.
- Ensure that the proportions of images in the code are preserved.
- Both the margin and padding in the code should be set to 0.
- Ensure that the generated code does not conflict with outer elements in terms of layout and style.
- The final return should be the complete HTML code, that is, including the above-mentioned framework and the code you generated and embedded into the <body> of the framework.

(2) **Appearance and Layout Consistency:**

- Ensure the app looks exactly like the screenshot, including the position, hierarchy, and content of all elements.
- The generated HTML elements and Tailwind classes should match those in the screenshot, ensuring that text, colors, fonts, padding, margins, borders, and other styles are perfectly aligned.

(3) **Content Consistency:**

- Use the exact text from the screenshot, ensuring the content of every element matches the image.
- For images, use placeholder images from https://placehold.co and include a detailed description in the alt text for AI-generated images.

(4) **No Comments or Placeholders:**

- Do not add comments like "<!– Add other navigation links as needed –>" or "<!– ... other news items ... –>". Write the full, complete code for each element.

(5) **Libraries to Use:**

- Use the following libraries:
  - Google Fonts: Use the relevant fonts from the screenshot.

Process Steps:

(1) **Analyze the Section:** Based on the provided screenshot, analyze a specific section of the webpage (such as the header, footer, card, form, etc.). Break down all the elements in this section (e.g., text, images, buttons, etc.) and understand their relative positions and hierarchy.
(2) **Generate HTML Code:** Based on the analysis from Step 1, generate a complete HTML code snippet representing that specific section, ensuring all elements, positions, and styles match the screenshot.
(3) **Text Content Comparison:** Compare the generated HTML with the screenshot's text content to ensure accuracy. If there are any discrepancies or missing content, make corrections.
(4) **Color Comparison:** Compare the text color and background color in the generated HTML with those in the screenshot. If they don't match, adjust the Tailwind classes and styles to reflect the correct colors.
(5) **Background and Other Style Comparison:** Ensure the background colors, borders, padding, margins, and other styles in the generated HTML accurately reflect the design shown in the screenshot.
(6) **Final Integration:** After reviewing and refining the previous steps, ensure that the generated HTML code is complete and perfectly matches the specific section of the screenshot.

Code Format: Please return the complete HTML code.

Figure 7: Prompt for block-wise code synthesis.